# Google Cloud Storage (GCS)

## Objective:

To ensure the transformed datasets are securely accessible and ready for further use, this step involves uploading them to a Google Cloud Storage (GCS) bucket. The key goals include:

- **Data Security**: Ensure that the uploaded datasets are protected through proper access controls, encryption at rest and in transit, and IAM policies that restrict access to authorized users only
- **Integration with Google Cloud Ecosystem**: Enable seamless connectivity with other Google services, such as BigQuery for advanced analytics, Looker Studio for visual reporting, and Vertex AI for machine learning applications.
- **Pipeline Enablement**: Support end-to-end data pipelines by making datasets available for:
  - Business intelligence dashboards (Power Bi)
  - Scheduled updates or data refreshes through automated workflow.

## Tools & Configuration:

1. **Platform**: Google Cloud Platform (GCP)
2. **Service**: Cloud Storage (GCS)
3. **Authentication**: GCP service account key (JSON)
4. **Upload Method**: Python script using google-cloud-storage SDK
5. **Destination Bucket**: gs://ppnj_data/files /

## Process Overview:

1. Authenticate using a service account key (credentials.json).
2. Initialize GCS client.
3. Define destination path in the GCS bucket.
4. Upload files in the list variable:
   - fact_sales.csv
   - dim_customers
   - dim_travel_package.csv
   - dim_travel_guide.csv
   - dim_feedback.csv
   - dim_refund.csv
5. Verify upload by listing objects in the bucket

## Limitations:

The Google Cloud Platform (GCP) used in this project is owned by the project author and is not the official GCP account of the company

# Sample Upload Script (Python)

```
import os
from google.cloud import storage

# path to GCP credentials
os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = r"C:\Users\User\Desktop\GCP\ppnj-463412-
13c39ea0960c.json"

# function to upload files to GCS
def upload_to_gcs(bucket_name, source_file, destination_blob):
    client = storage.Client()
    bucket = client.bucket(bucket_name)
    blob = bucket.blob(destination_blob)

    blob.upload_from_filename(source_file)
    print(f"Uploaded {source_file} to gs://{bucket_name}/{destination_blob}")


# Files to upload
files_to_upload = [
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End
Project\PPNJ\data_creation\dim_customer.csv",
        "destination_blob": "files/dim_customer.csv"
    },
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End
Project\PPNJ\data_creation\dim_feedback.csv",
        "destination_blob": "files/dim_feedback.csv"
    },
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End Project\PPNJ\data_creation\dim_refund.csv",
        "destination_blob": "files/dim_refund.csv"
    },
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End Project\PPNJ\data_creation\fact_sales.csv",
        "destination_blob": "files/fact_sales.csv"
    },
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End
Project\PPNJ\data_creation\dim_travel_guide.csv",
        "destination_blob": "files/dim_travel_guide.csv"
    },
    {
        "source_file": r"C:\Users\User\Desktop\Data Analyst\End To End
Project\PPNJ\data_creation\dim_travel_package.csv",
        "destination_blob": "files/dim_travel_package.csv"
    }
]

# Upload all files to GCS
for file in files_to_upload:
    upload_to_gcs("ppnj_data", file['source_file'], file['destination_blob'])
```

## Files Uploaded to GCS

| File Name | GCS Path |
|---|---|
| fact_sales.csv | gs://ppnj_data/files/fact_sales.csv |
| dim_customers.csv | gs://ppnj_data/files/dim_customer.csv |
| dim_travel_package.csv | gs://ppnj_data/files/dim_travel_package.csv |
| dim_travel_guide.csv | gs://ppnj_data/files/dim_travel_guide.csv |
| dim_feedback.csv | gs://ppnj_data/files/dim_feedback.csv |
| dim_refund.csv | gs://ppnj_data/files/dim_refund.csv |

## Summary:

1. All files are stored in GCS with private access by default.
2. It can be easily connected to BigQuery, Data Studio, or external dashboards.
3. Upload is fully scripted and repeatable, supporting automation.